Bachelor Thesis Exposé

Design and Implementation of an IoT-Based System for Smart Home Simulation

Marko Ankic

Introduction

The Internet of Things (IoT) has revolutionized the way we interact with our living spaces, enabling unprecedented levels of automation and control. Smart home systems represent one of the most prominent applications of IoT technology, allowing users to remotely monitor and control various household devices such as lighting, climate control, security systems, and entertainment systems. As the adoption of smart home technologies continues to grow, there is an increasing need for robust, scalable, and user-friendly platforms that can integrate diverse IoT devices into a cohesive ecosystem.

Research Objectives

The primary objective of this bachelor thesis is to design, implement, and evaluate a comprehensive IoT-based smart home simulator that demonstrates the fundamental principles of distributed device management, real-time communication, and web-based user interfaces. Specific objectives include:

1. **Design a scalable three-tier architecture** consisting of IoT devices, a backend server, and a web-based dashboard.
2. **Implement multiple device types** using MicroPython on Raspberry Pi Pico 2W microcontrollers.
3. **Develop a RESTful API backend** with real-time WebSocket communication for device state management.
4. **Create an intuitive web interface** for device control, room organization, and automation scenarios.
5. **Implement automatic device discovery** using UDP broadcast protocols.
6. **Ensure persistent data storage** using a relational database with flexible device property management.
7. **Demonstrate real-time synchronization** between physical devices and the web interface.

System Architecture and Design

The IoT Smart Home Simulator follows a **three-tier architecture** pattern, consisting of:

1. **Device Layer (Edge Tier):** Physical IoT devices implemented on Raspberry Pi Pico 2W microcontrollers running MicroPython.
2. **Backend Layer (Application Tier):** Flask-based REST API server with PostgreSQL database and Socket.IO for real-time communication.
3. **Presentation Layer (Client Tier):** Vue 3 single-page application (SPA) providing a responsive web dashboard.

This architecture ensures clear separation of concerns, scalability, and maintainability while demonstrating industry-standard design patterns for IoT systems.

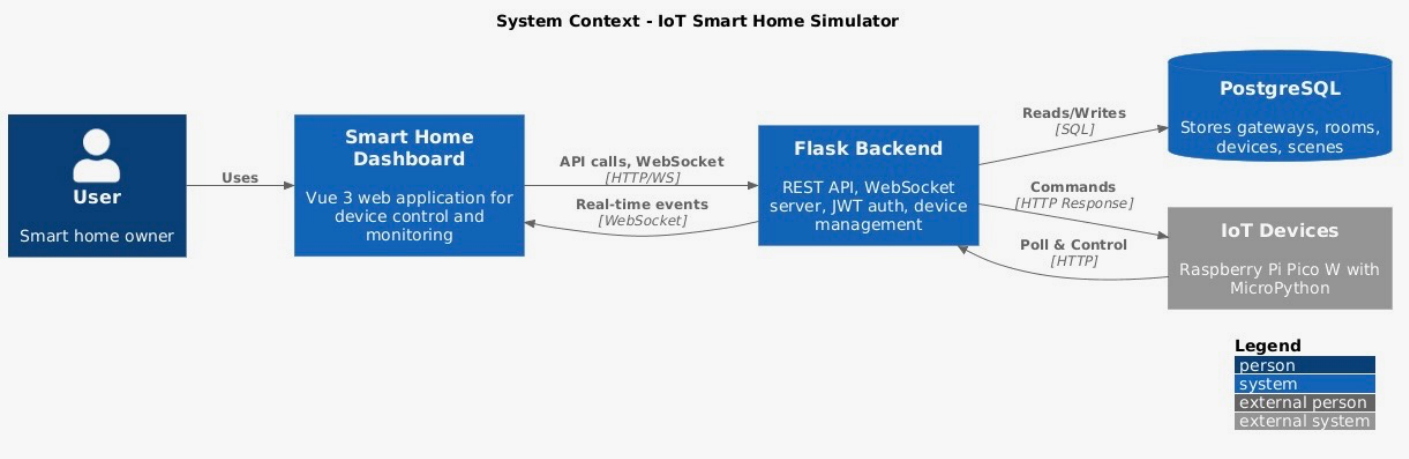


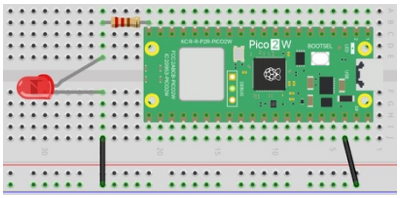
Figure 1 System Architecture - C4 Diagram

Device Layer Architecture

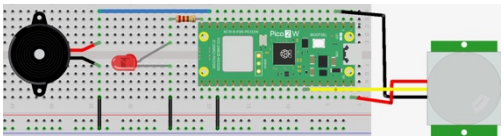
Devices are programmed using **MicroPython**, a lean implementation of Python optimized for microcontrollers. This choice provides: rapid prototyping capabilities, easy debugging and development, rich library ecosystem for hardware interfacing, low barrier to entry for developers.

The system implements **five distinct device types**, each demonstrating different IoT capabilities:

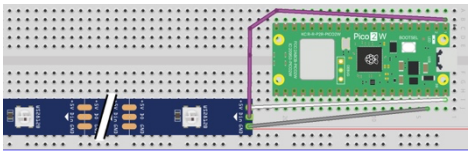
1. **Simple LED (simpleled)**
   - Basic binary state control (on/off)
   - Demonstrates fundamental device communication patterns
   - Polling-based state synchronization



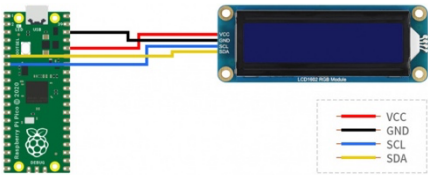
2. **Alarm Sensor (alarmSensor)**
   - Motion detection using PIR (Passive Infrared) sensor
   - Event-driven architecture with real-time notifications
   - Demonstrates sensor-to-server push communication



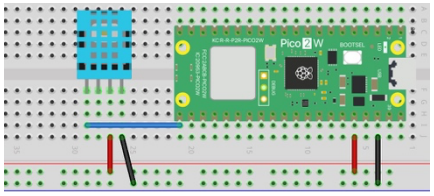
3. **RGB LED Strip (RGBStrip)**
   - Individually addressable WS2812 LED strip control
   - Complex property management (color, brightness, patterns)
   - Demonstrates handling of multi-dimensional device state



4. **LCD Display (LCDDisplay)**
   - I2C-based LCD screen control
   - Text message display and backlight control
   - Demonstrates output device management



5. **Weather Station (weatherStation)**
   - DHT11 temperature and humidity sensor integration
   - External API integration for weather forecasting
   - Periodic data collection and aggregation
   - Demonstrates sensor data processing pipeline



All devices implement a **common communication pattern**:

1. **WiFi Connection** – Devices connect to local WiFi network using stored credentials or interactive setup.
2. **Server Discovery** - UDP broadcast mechanism automatically locates the backend server.

3. **Device Registration** - HTTP POST request to '*/api/pair-device*' registers the device with the server.
4. **Continuous Polling** – Devices poll '*/api/devices/{type}/{id}/state*' endpoint at configurable intervals (typically 2 seconds).
5. **State Synchronization** – Server responds with updated device properties, which the device applies to its hardware.
6. **Availability Tracking** – Server monitors '*last_seen*' timestamp to determine device online/offline status.

This polling-based approach, while not the most bandwidth-efficient, provides reliable state synchronization, easy debugging and monitoring and no complex keep-alive or heartbeat mechanisms.

Backend Layer Architecture

The backend is built using:

- **Flask** – Lightweight web application framework
- **PostgreSQL** – Robust relational database with JSONB support
- **SQLAlchemy** – ORM for database abstraction
- **Flask-SocketIO** – WebSocket implementation for real-time communication
- **APSScheduler** – Background task scheduling for periodic jobs
- **PyJWT** – JSON Web Token authentication

Database Schema

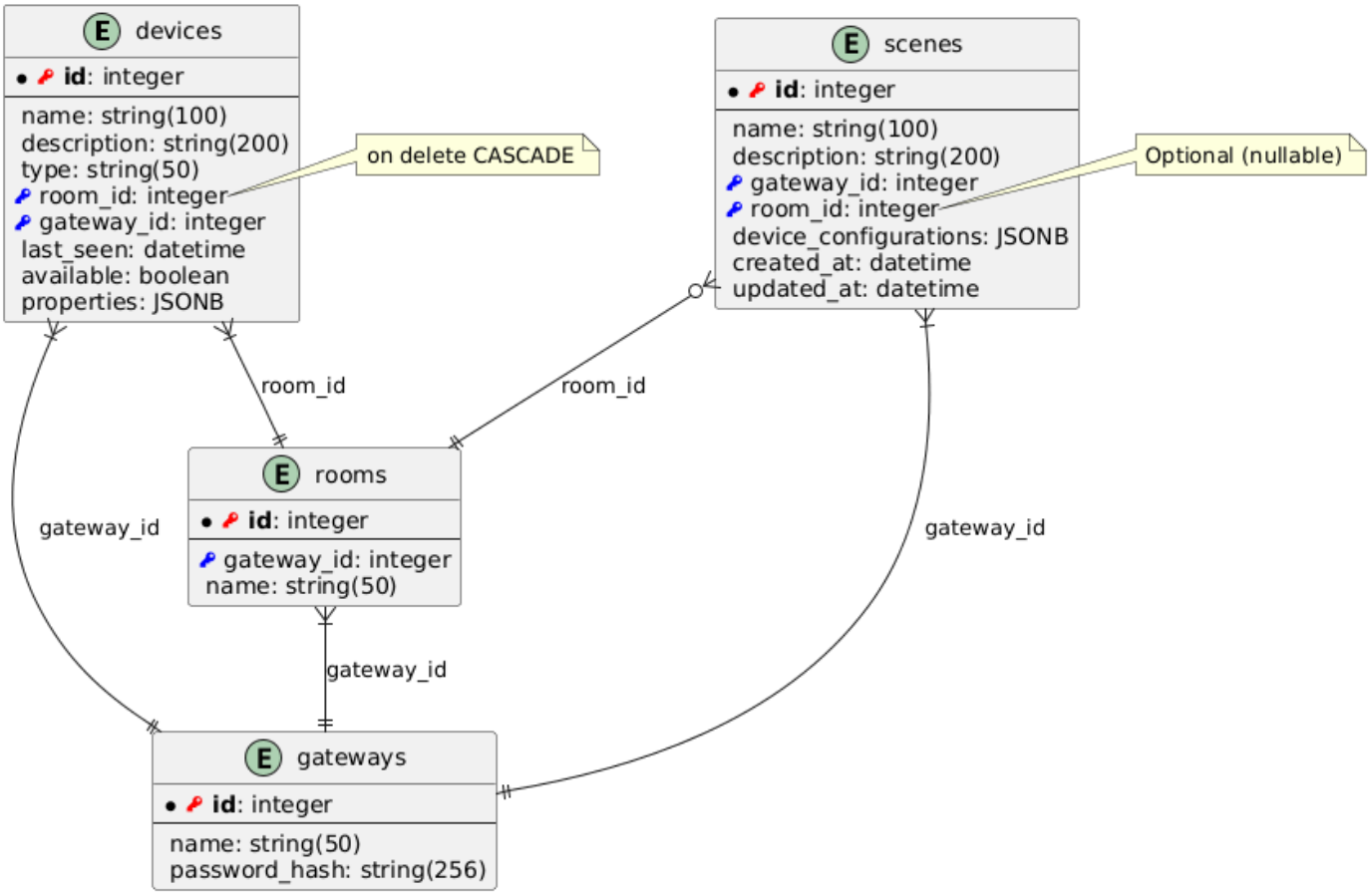


Figure 2 Data Base - ER Diagram

The system implements a **hierarchical data model** consisting of four primary entities:

1. Gateway - Represents the central hub of the smart home system.
2. Room - Logical grouping of devices by physical location.
3. Device – Represents individual IoT devices.
4. Scene – Automation rules combining multiple device actions.

The use of **PostgreSQL's JSONB** data type for device properties is a key design decision, providing: flexibility to add new device types without schema migrations, efficient indexing and querying of JSONB data, type safety through PostgreSQL's validation, compact storage for complex nested properties.

The system implements **Socket.IO** for bidirectional real-time communication. The backend uses a dual-process emission pattern:
1. API routes update database state.
2. Route calls HTTP POST to '*/emit-**' endpoints.
3. Emissions endpoints trigger SOCKET.IO broadcasts.
4. All connected clients receive updates instantly.

An **APScheduler background job** runs every two seconds to monitor device availability:
1. Query all devices from database
2. Check if '*last_seen*' timestamp is within 10-second threshold
3. Update 'available' status if changed
4. Emit '*device_status*' event to connected clients
This mechanism ensures that the web interface accurately reflects the real-time status of physical devices, even if they disconnect unexpectedly (power loss, network failure, etc.).

The backend implements **UDP broadcast listener**:
1. Listens for '*SMART_HOME_SERVER_DISCOVERY'* broadcast messages
2. Responds with *'SMART_HOME_SERVER:{port}'* to sender
3. Devices extract server IP and port for HTTP communication
This auto-discovery mechanism significantly simplifies device setup and eliminates need for manual server IP configuration on devices.

Presentation Layer Architecture

The frontend is built as a **modern single-page application (SPA)** using:
- **VueJS (CompositionAPI)** – Progressive JavaScript framework with reactive data binding
- **Vite** – Next-generation frontend build tool with HMR (Hot Module Replacement)
- **Pinia** – Official state management library for Vue
- **Axios** – HTTP client with interceptor support
- **Socket.IO Client** – WebSocket client for real-time updates
- **Tailwind CSS** – Utility-first CSS framework for responsive design

The frontend implements a **dynamic widget mapping system** to render device-specific controls.
Each device type has a dedicated Vue component:
- *SimpleLEDWidget* – Toggle button for LED control.
- *AlarmSensorWidget* – Status indicator and enable/disable control.
- *RGBStripWidget* – Color picker and brightness slider.
- *LCDDisplayWidget* – Text input and backlight toggle.
- *WeatherStationWidget* – Temperature, humidity and forecast display.

Widgets are **dynamically loaded** based on device type using *deviceWidgetMap* utility, ensuring extensibility for future device types.

The frontend establishes a **persistent Socket.IO connection** to the backend:
1. Connection establishes on application startup.
2. Listens for server-emitted events.
3. Updates Pinia store when events are received.
4. Vue's reactivity system automatically updates UI.
5. Reconnection logic handles network interruptions.
This architecture ensures that multiple users can view the same dashboard and see **synchronized device states in real-time.**

The application implements **JWT-based authentication** with automatic token refresh:
1. User submits credentials to */api/login*.
2. Server validates and returns access + refresh tokens.
3. Axios interceptor attaches access token to all requests.
4. On 401 response, interceptor attempts token refresh.
5. If refresh succeeds, original request is retried.
6. If refresh fails, user is redirected to login.
This pattern provides **seamless session management** without user intervention.

The frontend uses **TailwindCSS** for responsive layouts. The application is fully functional on devices ranging from smartphones to desktop monitors.

Future Work and Extensions

Although the developed system successfully demonstrates the core principles of IoT-based smart home architectures, several improvements could be explored in future work. One possible direction is the integration of additional device types such as smart thermostats, door locks, power meters, and camera modules, which would allow the system to simulate more complex smart home environments. The automation subsystem could also be extended with conditional rules, time-based triggers, and event-driven scenarios to enable more advanced interactions between devices.

From a technical perspective, the communication architecture could be optimized by replacing the current polling-based mechanism with more efficient protocols such as MQTT or WebSocket-based device communication. Additional security mechanisms, including encrypted communication and stronger device authentication, could further improve system robustness and reliability.

Finally, the system could be extended to support data analytics and machine learning techniques applied to collected sensor data. Such extensions could enable functionality such as usage pattern analysis, anomaly detection in device behavior, and energy consumption optimization, demonstrating how IoT-generated data can support intelligent smart home automation.

Conclusion

The IoT Smart Home Simulator successfully demonstrates that modern web technologies, affordable microcontrollers, and open-source software can be combined to create functional, extensible, and educational smart home systems. While not intended for production deployment, the implementation provides a solid foundation for understanding the architecture, communication patterns, and design decisions that underpin real-world IoT platforms.

The journey from initial concept to working system involved navigating challenges in hardware interfacing, network programming, real-time communication, database design, and user interface development. Each challenge provided valuable learning experiences and insights that are reflected in the final implementation.

As smart home technology continues to evolve and proliferate, systems like this serve as accessible entry points for newcomers to the field and experimental platforms for researchers pushing the boundaries of what's possible in connected home environments. The future of smart homes is bright, and projects like this contribute to making that future more open, understandable, and accessible to all.

This exposé provides a comprehensive overview of the bachelor thesis project "Design and Implementation of an IoT-Based System for Smart Home Simulation". For complete implementation details, please refer to the source code repository and accompanying thesis documentation.